Laser-pulse heating of an ENIG stack

Scientific coding work sample: problem specification, Python reference solution and hidden tests. Alvin Kigondu, October 2026.

This is the conduction stage of a laser-ablation and LIBS model I am building for ENIG-finished PCB contacts (gold over nickel over copper). I wrote it up in the form the role describes: a self-contained problem, a reference solution, and hidden tests whose expected values are computed independently of that solution. Section 5 covers how I checked the tests themselves, including a bug they found in my own code.

1. Problem specification

Compute the temperature field in a stack of metal layers heated by one Gaussian laser pulse, below the melting point. One spatial coordinate x (m) runs from the irradiated surface ($x = 0$) into the material. Layers are listed from the surface down; the total thickness is L. Time starts at $t = 0$ and the pulse peaks at t_{peak}. SI units throughout, temperatures in kelvin.

Inside layer j (constant properties, Fourier conduction) with the laser source:

```
ρ_j c_j ∂T/∂t = k_j ∂²T/∂x² + S(x,t)

S(x,t) = F · g(t) · p(x)
g(t)   = exp(−(t − t_peak)² / 2σ²) / (σ√(2π)),     σ = FWHM / (2√(2 ln 2))
p(x)   = exp(−x/δ) / (δ (1 − exp(−L/δ))),           0 ≤ x ≤ L
```

F is the absorbed fluence (J m^{-2}, net of reflection). Both g and p integrate to 1, so the energy deposited by time t is $F[\Phi((t-t_{peak})/\sigma) - \Phi(-t_{peak}/\sigma)]$. At each interface, temperature and heat flux $k\partial T/\partial x$ are continuous (perfect contact). Both faces are adiabatic ($\partial T/\partial x = 0$); radiation and convection are neglected. Initially $T(x, 0) = T_0$.

Function to implement (NumPy and SciPy only):

```
simulate_stack_heating(layers, absorbed_fluence, pulse_fwhm, t_peak,
                       penetration_depth, t_out, probe_depths, T0=300.0) -> ndarray
```

Argument	Type and units	Constraint and meaning
layers	list of dict	Non-empty. Keys thickness (m), k (W m^{-1} K^{-1}), rho (kg m^{-3}), cp (J kg^{-1} K^{-1}), all positive and finite. Surface layer first.
absorbed_fluence	float, J m^{-2}	≥ 0. F.
pulse_fwhm	float, s	> 0.
t_peak	float, s	≥ 3 × FWHM, so the energy cut off before $t = 0$ is below 10^{-11} of F.
penetration_depth	float, m	> 0. δ, one value for the whole stack.
t_out	array (n_t,), s	Strictly increasing, all > 0.
probe_depths	array (n_x,), m	Each in [0, L].
T0	float, K	> 0.
Returns	ndarray float64 (n_t, n_x)	Element [i, j] is T(probe_depths[j], t_out[i]) in K. Raises ValueError for any input outside the constraints.

Valid regime and approximations

- Peak temperature below the lowest melting point in the stack (Au 1337 K, Cu 1358 K, Ni 1728 K): no phase change, no ablation.
- Pulse FWHM of about 1 ns or longer. Picosecond pulses need a two-temperature (electron and lattice) model.
- Constant k, ρ, c_p per layer; one exponential absorption depth. In gold, ballistic electrons spread the deposited energy deeper than the 13 nm optical depth at 1064 nm; that is ignored.
- No thermal boundary resistance at interfaces.

Reference stack and pulse

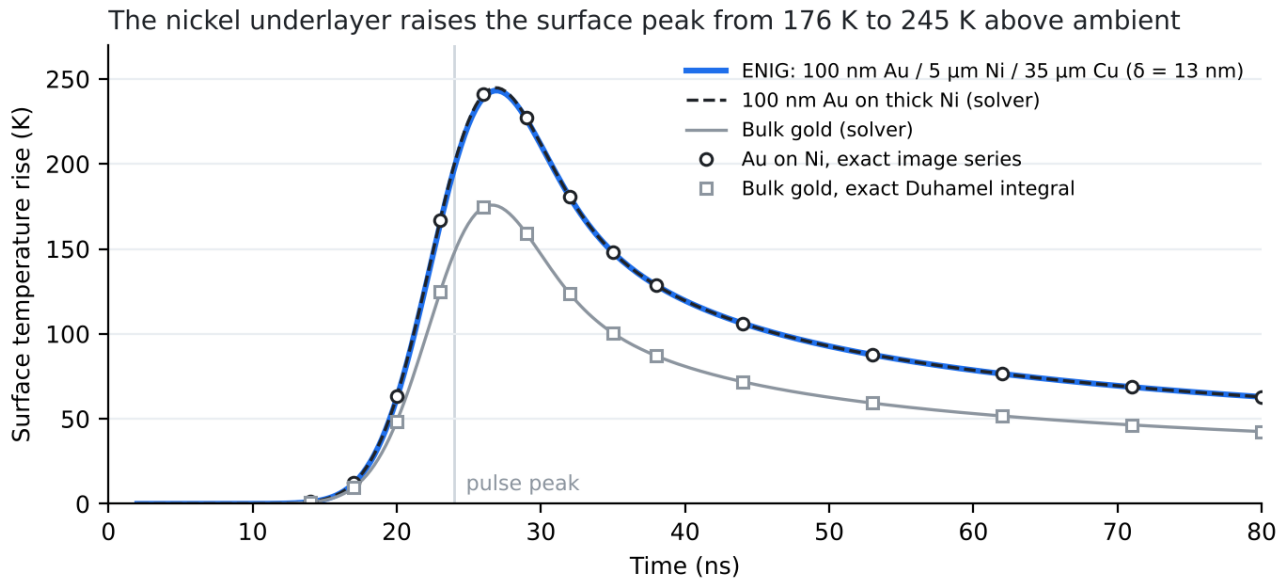
Layer	Thickness	k (W m^{-1} K^{-1})	ρ (kg m^{-3})	c_p (J kg^{-1} K^{-1})
Au	100 nm	317	19 300	129
Ni	5 µm	90.9	8 900	444
Cu	35 µm	401	8 960	385

Nd:YAG at 1064 nm, FWHM 8 ns, t_{peak} = 24 ns, δ = 13 nm, absorbed fluence 500 J m^{-2} (0.05 J cm^{-2}; about 1.7 J cm^{-2} incident at 97% reflectance).

2. Reference solution

- Vertex-centred finite volumes on a graded mesh. Every layer interface is a node, so no segment crosses an interface; an interface node's heat capacity is split between the two materials. This gives flux continuity under perfect contact without special-casing.
- The first cell is min(δ/5, √(α_{min}·FWHM)/10). Cells grow by 8% per step up to L/300 and the step size carries across interfaces.
- The source is integrated exactly over each control volume, so the scheme deposits exactly the analytic energy. With adiabatic faces it conserves energy to the time integrator's tolerance.
- The system is stiff (sub-nanometre to sub-micrometre cells), so it uses implicit BDF with the constant sparse Jacobian, a step cap of FWHM/20 while the pulse is on, and free steps after it.
- Probe temperatures are linear interpolations between nodes. The reference stack runs in about 0.2 s.

3. Result and validation



Lines are the reference solution; markers are values computed without it. For bulk gold the markers come from the exact Duhamel integral for a semi-infinite solid. For a 100 nm gold film on nickel they come from the exact image-series solution (reflection coefficient Γ = (e_{Au} − e_{Ni})/(e_{Au} + e_{Ni}) = 0.19, e = √(kρc_p)). Heat that would spread into more gold instead meets nickel, which absorbs it more slowly, so the ENIG surface runs about 70 K hotter at peak. The copper under 5 µm of nickel has no effect on this timescale.

4. Hidden tests

25 test cases (pytest), all passing. The last column is how far the reference solution sits from each expected value; the gap between it and the tolerance is the room left for valid alternative methods (finite difference, finite element, other time integrators).

Test	Expected value from	Tolerance	Reference error
Thick Cu, surface and two depths, 8 ns pulse	Duhamel integral (scipy quad, algebraic weight for the 1/√ singularity)	1% of peak rise	0.10%
Same Cu split into three layers at the ENIG depths	Duhamel integral	1% of peak rise	0.05%

Test	Expected value from	Tolerance	Reference error
1 ns pulse on gold, outputs from 2.5 ns to 1 μs (stiff)	Duhamel integral	1% of peak rise	0.19%
100 nm Au on thick Ni, surface and interface	Exact image series	1% of peak rise	0.09%
Energy balance, ENIG, mid-pulse and after	Deposited energy in closed form	0.5%	7e-8
Late-time uniform temperature, ENIG	$F / \Sigma \rho c_p d$	0.1%	2e-8
Linearity in F; rise independent of T0	Exact factor of 2	1e-4	3e-9
Zero fluence	T0 everywhere	1e-9 K	exactly T0
No point ever below T0 (40 times × 81 depths)	Maximum principle	−1e-6 K	never below T0
Ni underlayer raises the surface peak over bulk Au	Physical expectation	> 10 K	69 K higher
300 output times vs 4 of them requested alone	Self-consistency	1e-4	identical
Output type and shape	Spec	exact	pass
12 invalid inputs (empty, negative, unsorted, out of range...)	Spec	ValueError	pass

5. Checking the tests

Passing tests only mean something if the tests can fail. I planted plausible bugs in the solution one at a time and reran the suite. The count is how many test cases each bug breaks.

Planted defect	Tests failed
Pulse width used as sigma (FWHM-to-sigma factor dropped)	6
Heat capacity uses density only (cp left out)	5
Each segment takes the conductivity of the layer below	2
Conductivity averaged across neighbouring segments	1
Source sampled at nodes instead of integrated per cell	3
Uniform mesh at L/300 (no grading near the surface)	4
Back face held at T0 instead of adiabatic	1
Probe read at nearest node instead of interpolated	3
Surface node given a full cell of heat capacity	0
No step-size cap while the pulse is on	0
First cell 50 times larger	0

What this showed:

- In the first round, the conductivity-swap defect survived every quantitative test and was caught only by the qualitative nickel check. All my analytic cases were single-material, so nothing measured heat flow between two different metals. I added the gold-on-nickel image-series test to close that gap.
- The last three defects pass, and should. The surface half-cell is at most about a nanometre thick, so its capacity barely matters; BDF picks small enough steps without the cap; and a first cell 50 times larger still resolves the diffusion length. The largest shift among them is 0.13% of the peak, and that one moves toward the analytic value, because the analytic cases assume a pure surface flux and a coarser first cell behaves more like one. A test strict enough to fail these would also fail correct solutions written a different way.
- Making the figure exposed a real bug in the reference solution. The solver splits time at t_{peak} + 4 FWHM, and an output time landing exactly on that split was passed to both segments, so SciPy rejected the unsorted output grid. The original tests never requested that time. I wrote the dense-grid test first, confirmed it failed, then fixed the split. In review terms the defect sat in the solution code, not the spec

or the expected values.

Appendix A. stack_heating.py (reference solution)

```
"""Reference solution: laser-pulse heating of a layered metal stack.

Method
------
Vertex-centred finite volumes on a graded 1-D mesh. Every layer interface is a
node, so no segment crosses an interface and the conductance between two nodes
uses a single conductivity. The control volume of an interface node is split
between the two materials, which gives flux continuity with perfect contact.

The laser source is integrated exactly over each control volume, so the
discrete scheme deposits exactly F * integral(g) of energy. With adiabatic
boundaries the semi-discrete system conserves energy exactly; only the time
integrator's tolerance limits the energy balance.

The system C dT/dt = -K T + q(t) is stiff (cells range from sub-nanometre to
sub-micrometre), so it is integrated with the implicit BDF method, a constant
sparse Jacobian, and a step cap during the pulse so the source is resolved.
"""

from __future__ import annotations

import math

import numpy as np
from scipy import sparse
from scipy.integrate import solve_ivp

_REQUIRED_KEYS = ("thickness", "k", "rho", "cp")


def _check_inputs(layers, F, fwhm, t_peak, delta, t_out, probes, T0):
    if not isinstance(layers, (list, tuple)) or len(layers) == 0:
        raise ValueError("layers must be a non-empty list of dicts")
    for i, layer in enumerate(layers):
        for key in _REQUIRED_KEYS:
            if key not in layer:
                raise ValueError(f"layer {i} is missing '{key}'")
            v = float(layer[key])
            if not (math.isfinite(v) and v > 0):
                raise ValueError(f"layer {i}: '{key}' must be positive and finite")
    for name, v in (("pulse_fwhm", fwhm), ("penetration_depth", delta), ("T0", T0)):
        if not (math.isfinite(v) and v > 0):
            raise ValueError(f"{name} must be positive and finite")
    if not (math.isfinite(F) and F >= 0):
        raise ValueError("absorbed_fluence must be finite and >= 0")
    if not (math.isfinite(t_peak) and t_peak >= 3.0 * fwhm * (1.0 - 1e-12)):
        raise ValueError("t_peak must be >= 3 * pulse_fwhm")
    if t_out.ndim != 1 or t_out.size == 0:
        raise ValueError("t_out must be a non-empty 1-D array")
    if not np.all(np.isfinite(t_out)) or t_out[0] <= 0 or np.any(np.diff(t_out) <= 0):
        raise ValueError("t_out must be strictly increasing and > 0")
    L = sum(float(layer["thickness"]) for layer in layers)
    if probes.ndim != 1 or probes.size == 0:
        raise ValueError("probe_depths must be a non-empty 1-D array")
    if not np.all(np.isfinite(probes)) or probes.min() < 0 or probes.max() > L * (1.0 + 1e-12):
        raise ValueError("probe_depths must lie in [0, L]")


def _build_mesh(edges, h0, h_max, growth=1.08):
    """Graded node positions: fine at the surface, growing geometrically.

    The step size carries over from one layer to the next. Each layer ends
    exactly on its interface; a remainder between h and 1.5 h is split in two
    so no cell is much larger than the local target size.
    """
    nodes = [0.0]
    h = h0
    for a, b in zip(edges[:-1], edges[1:]):
        x = a
        while b - x > 1.5 * h:
            x += h
            nodes.append(x)
            h = min(h * growth, h_max)
        if b - x > h:
            nodes.append(x + 0.5 * (b - x))
        nodes.append(b)
    return np.asarray(nodes)


def simulate_stack_heating(layers, absorbed_fluence, pulse_fwhm, t_peak,
                           penetration_depth, t_out, probe_depths, T0=300.0):
    F = float(absorbed_fluence)
    fwhm = float(pulse_fwhm)
    t_peak = float(t_peak)
    delta = float(penetration_depth)
    T0 = float(T0)
```

```
t_out = np.asarray(t_out, dtype=float)
probes = np.asarray(probe_depths, dtype=float)
_check_inputs(layers, F, fwhm, t_peak, delta, t_out, probes, T0)

thick = np.array([float(l["thickness"]) for l in layers])
k = np.array([float(l["k"]) for l in layers])
rc = np.array([float(l["rho"]) * float(l["cp"]) for l in layers])
edges = np.concatenate([[0.0], np.cumsum(thick)])
L = edges[-1]

# Mesh: resolve the absorption depth and the shortest thermal diffusion
# length during the pulse at the surface; cap the coarsest cell at L/300.
alpha_min = float(np.min(k / rc))
h0 = min(delta / 5.0, math.sqrt(alpha_min * fwhm) / 10.0, L / 300.0)
x = _build_mesh(edges, h0, L / 300.0)
n = x.size
dx = np.diff(x)                                  # segment lengths, n-1
xmid = 0.5 * (x[:-1] + x[1:])
seg_layer = np.clip(np.searchsorted(edges, xmid, side="right") - 1, 0, len(layers) - 1)

# Heat capacity per node (J m^-2 K^-1): half of each adjacent segment.
seg_cap = rc[seg_layer] * dx
C = np.zeros(n)
C[:-1] += 0.5 * seg_cap
C[1:] += 0.5 * seg_cap

# Conductance between neighbouring nodes (W m^-2 K^-1).
G = k[seg_layer] / dx
main = np.zeros(n)
main[:-1] += G
main[1:] += G
K = sparse.diags([-G, main, -G], [-1, 0, 1], format="csc")
A = sparse.diags(1.0 / C) @ K                    # dT/dt = -A T + q/C
A = A.tocsc()

# Fraction of the absorbed energy landing in each control volume (exact).
faces = np.concatenate([[0.0], xmid, [L]])
norm = -math.expm1(-L / delta)
w = (np.exp(-faces[:-1] / delta) - np.exp(-faces[1:] / delta)) / norm
src = F * w / C                                  # K per unit of g(t)

sigma = fwhm / (2.0 * math.sqrt(2.0 * math.log(2.0)))
gnorm = 1.0 / (sigma * math.sqrt(2.0 * math.pi))

def g(t):
    return gnorm * math.exp(-0.5 * ((t - t_peak) / sigma) ** 2)

def rhs(t, T):
    return -(A @ T) + src * g(t)

jac = -A
# Solve for the rise above T0; the problem is linear so this is exact.
y0 = np.zeros(n)
t_end = float(t_out[-1])
t_pulse_end = t_peak + 4.0 * fwhm
rtol, atol = 1e-8, 1e-8

results = []
if t_end <= t_pulse_end:
    sol = solve_ivp(rhs, (0.0, t_end), y0, method="BDF", jac=jac, t_eval=t_out,
                    max_step=fwhm / 20.0, rtol=rtol, atol=atol)
    if not sol.success:
        raise RuntimeError(sol.message)
    results.append(sol.y)
else:
    # Strict split: a requested time equal to t_pulse_end belongs to the
    # second segment, otherwise t_eval would hold it twice.
    early = t_out[t_out < t_pulse_end]
    late = t_out[t_out >= t_pulse_end]
    sol1 = solve_ivp(rhs, (0.0, t_pulse_end), y0, method="BDF", jac=jac,
                     t_eval=np.concatenate([early, [t_pulse_end]]),
                     max_step=fwhm / 20.0, rtol=rtol, atol=atol)
    if not sol1.success:
        raise RuntimeError(sol1.message)
    results.append(sol1.y[:, :-1])
    sol2 = solve_ivp(rhs, (t_pulse_end, t_end), sol1.y[:, -1], method="BDF",
                     jac=jac, t_eval=late, rtol=rtol, atol=atol)
    if not sol2.success:
        raise RuntimeError(sol2.message)
    results.append(sol2.y)

rise = np.concatenate(results, axis=1).T         # (n_t, n_nodes)
out = np.empty((t_out.size, probes.size))
for i in range(t_out.size):
    out[i] = T0 + np.interp(np.minimum(probes, L), x, rise[i])
return out
```

Appendix B. test_stack_heating.py (hidden tests)

```
"""Hidden tests for simulate_stack_heating.

Expected values come from sources independent of the reference solution:
- the Duhamel integral for a semi-infinite solid under a surface heat flux,
  evaluated with scipy.integrate.quad;
- the exact image-series solution for a film on a semi-infinite substrate,
  built from that same integral;
- energy conservation (deposited energy is known in closed form);
- the uniform temperature reached once an adiabatic stack equilibrates;
- linearity of the heat equation in the source.
"""

import math

import numpy as np
import pytest
from scipy.integrate import quad
from scipy.special import ndtr

from stack_heating import simulate_stack_heating

AU = dict(thickness=100e-9, k=317.0, rho=19300.0, cp=129.0)
NI = dict(thickness=5e-6, k=90.9, rho=8900.0, cp=444.0)
CU = dict(thickness=35e-6, k=401.0, rho=8960.0, cp=385.0)
ENIG = [AU, NI, CU]

F_REF = 500.0          # J m^-2 absorbed
FWHM_REF = 8e-9        # s
TPEAK_REF = 24e-9      # s
DELTA_AU = 13e-9       # m, optical penetration depth of Au at 1064 nm


def _sigma(fwhm):
    return fwhm / (2.0 * math.sqrt(2.0 * math.log(2.0)))


def semi_infinite_rise(x, t, F, fwhm, t_peak, k, rho, cp):
    """Temperature rise in a semi-infinite solid with surface flux F*g(t).

    T(x,t) - T0 = 1/(e sqrt(pi)) * int_0^t q(s) exp(-x^2/(4 a (t-s))) / sqrt(t-s) ds,
    e = sqrt(k rho cp), a = k/(rho cp). Integrated with quad, independent of
    any mesh or time stepper.
    """
    sig = _sigma(fwhm)
    e = math.sqrt(k * rho * cp)
    a = k / (rho * cp)

    def q(s):
        return F * math.exp(-0.5 * ((s - t_peak) / sig) ** 2) / (sig * math.sqrt(2 * math.pi))

    lo = max(0.0, t_peak - 12 * sig)
    hi = t_peak + 12 * sig
    if t <= lo:
        return 0.0
    if t <= hi:
        # (t - s)^(-1/2) singularity at the upper end handled by an algebraic weight.
        def f(s):
            return q(s) * math.exp(-x * x / (4 * a * (t - s))) if s < t else (q(s) if x == 0 else 0.0)
        val, _ = quad(f, lo, t, weight="alg", wvar=(0.0, -0.5), limit=400)
    else:
        def f(s):
            return q(s) * math.exp(-x * x / (4 * a * (t - s))) / math.sqrt(t - s)
        val, _ = quad(f, lo, hi, limit=400, points=[t_peak])
    return val / (e * math.sqrt(math.pi))


def _analytic_grid(times, depths, material, F, fwhm, t_peak):
    return np.array([[semi_infinite_rise(x, t, F, fwhm, t_peak,
                                         material["k"], material["rho"], material["cp"])
                      for x in depths] for t in times])


def _assert_close_to_peak(T, T0, expected_rise, frac=0.01):
    tol = frac * np.max(np.abs(expected_rise))
    err = np.abs((T - T0) - expected_rise)
    assert np.max(err) <= tol, f"max error {np.max(err):.4g} K exceeds {tol:.4g} K"


# 1. Standard regime, limiting case: thick homogeneous Cu behaves as a
#    semi-infinite solid; very small delta makes the source a surface flux.
def test_semi_infinite_copper_surface_and_depth():
    cu_thick = dict(CU, thickness=200e-6)
    times = np.array([16e-9, 24e-9, 30e-9, 50e-9, 200e-9])
    depths = np.array([0.0, 0.5e-6, 2e-6])
    T = simulate_stack_heating([cu_thick], F_REF, FWHM_REF, TPEAK_REF, 0.5e-9,
                               times, depths, T0=300.0)
```

```
    expected = _analytic_grid(times, depths, CU, F_REF, FWHM_REF, TPEAK_REF)
    _assert_close_to_peak(T, 300.0, expected)


# 2. Interface bookkeeping: one material split into three layers at the ENIG
#    interface depths must give the same answer as the homogeneous solid.
def test_split_identical_layers_match_homogeneous():
    layers = [dict(CU, thickness=100e-9), dict(CU, thickness=5e-6), dict(CU, thickness=194.9e-6)]
    times = np.array([20e-9, 30e-9, 100e-9, 1e-6])
    depths = np.array([0.0, 100e-9, 5.1e-6, 10e-6])
    T = simulate_stack_heating(layers, F_REF, FWHM_REF, TPEAK_REF, 0.5e-9, times, depths)
    expected = _analytic_grid(times, depths, CU, F_REF, FWHM_REF, TPEAK_REF)
    _assert_close_to_peak(T, 300.0, expected)


# 3. Numerically hard case: 1 ns pulse on gold, steep surface gradient,
#    output times spanning three decades.
def test_short_pulse_gold_stiff_case():
    au_thick = dict(AU, thickness=100e-6)
    times = np.array([2.5e-9, 3e-9, 3.5e-9, 5e-9, 1e-6])
    depths = np.array([0.0, 0.2e-6, 1e-6])
    T = simulate_stack_heating([au_thick], 200.0, 1e-9, 3e-9, 0.5e-9, times, depths)
    expected = _analytic_grid(times, depths, AU, 200.0, 1e-9, 3e-9)
    _assert_close_to_peak(T, 300.0, expected)


def _stored_energy(T, T0, depths, layers):
    """Integral of rho*cp*(T - T0) dx using the trapezoid rule within each layer."""
    edges = np.concatenate([[0.0], np.cumsum([l["thickness"] for l in layers])])
    total = 0.0
    for j, layer in enumerate(layers):
        m = (depths >= edges[j]) & (depths <= edges[j + 1])
        total += layer["rho"] * layer["cp"] * np.trapezoid(T[m] - T0, depths[m])
    return total


def film_on_substrate_rise(x_where, t, F, fwhm, t_peak, d, film, substrate, n_terms=40):
    """Exact image-series solution for a film of thickness d on a semi-infinite
    substrate, surface flux F*g(t), perfect contact.

    From the Laplace-domain solution, with Gamma = (e1 - e2) / (e1 + e2) and
    u(x, t) the semi-infinite response of the film material:
      surface:   T(0, t) = u(0, t) + 2 * sum_{n>=1} Gamma^n u(2 n d, t)
      interface: T(d, t) = (1 + Gamma) * sum_{n>=0} Gamma^n u((2n+1) d, t)
    """
    e1 = math.sqrt(film["k"] * film["rho"] * film["cp"])
    e2 = math.sqrt(substrate["k"] * substrate["rho"] * substrate["cp"])
    gam = (e1 - e2) / (e1 + e2)

    def u(x):
        return semi_infinite_rise(x, t, F, fwhm, t_peak, film["k"], film["rho"], film["cp"])

    if x_where == "surface":
        return u(0.0) + 2.0 * sum(gam ** n * u(2 * n * d) for n in range(1, n_terms))
    return (1.0 + gam) * sum(gam ** n * u((2 * n + 1) * d) for n in range(n_terms))


# 3b. Two different materials: 100 nm Au on thick Ni against the exact
#     image-series solution, at the surface and at the Au/Ni interface.
def test_gold_film_on_nickel_matches_image_series():
    d = 100e-9
    layers = [dict(AU, thickness=d), dict(NI, thickness=200e-6)]
    times = np.array([16e-9, 24e-9, 30e-9, 50e-9, 100e-9])
    T = simulate_stack_heating(layers, F_REF, FWHM_REF, TPEAK_REF, 0.5e-9, times, [0.0, d])
    expected = np.array([[film_on_substrate_rise(w, t, F_REF, FWHM_REF, TPEAK_REF, d, AU, NI)
                          for w in ("surface", "interface")] for t in times])
    _assert_close_to_peak(T, 300.0, expected)


# 4. Conservation: with adiabatic faces, stored energy equals deposited energy,
#    mid-pulse and after the pulse.
@pytest.mark.parametrize("t", [20e-9, 100e-9])
def test_energy_balance_enig(t):
    L = sum(l["thickness"] for l in ENIG)
    edges = np.cumsum([l["thickness"] for l in ENIG])
    depths = np.unique(np.concatenate([[0.0], np.geomspace(1e-11, L, 20000), edges]))
    depths = depths[depths <= L]
    T = simulate_stack_heating(ENIG, F_REF, FWHM_REF, TPEAK_REF, DELTA_AU, [t], depths)[0]
    sig = _sigma(FWHM_REF)
    deposited = F_REF * (ndtr((t - TPEAK_REF) / sig) - ndtr(-TPEAK_REF / sig))
    stored = _stored_energy(T, 300.0, depths, ENIG)
    assert abs(stored - deposited) / deposited < 5e-3


# 5. Late-time limit: the adiabatic stack relaxes to one uniform temperature
#    set by total heat capacity.
def test_late_time_equilibrium_enig():
    depths = np.array([0.0, 50e-9, 100e-9, 2e-6, 5.1e-6, 20e-6, 40.1e-6])
    T = simulate_stack_heating(ENIG, F_REF, FWHM_REF, TPEAK_REF, DELTA_AU, [3e-4], depths)[0]
```

```
    heat_cap = sum(l["rho"] * l["cp"] * l["thickness"] for l in ENIG)
    rise = F_REF / heat_cap
    assert np.max(np.abs((T - 300.0) - rise)) / rise < 1e-3


# 6. Linearity in the source, and the rise does not depend on T0.
def test_linear_in_fluence_and_independent_of_T0():
    times = [24e-9, 60e-9, 1e-6]
    depths = [0.0, 100e-9, 3e-6]
    T1 = simulate_stack_heating(ENIG, 250.0, FWHM_REF, TPEAK_REF, DELTA_AU, times, depths, T0=300.0)
    T2 = simulate_stack_heating(ENIG, 500.0, FWHM_REF, TPEAK_REF, DELTA_AU, times, depths, T0=77.0)
    r1, r2 = T1 - 300.0, T2 - 77.0
    assert np.max(np.abs(r2 - 2.0 * r1)) / np.max(r2) < 1e-4


# 7. Boundary behaviour: no energy in, no change; heating never cools anything.
def test_zero_fluence_stays_at_T0():
    T = simulate_stack_heating(ENIG, 0.0, FWHM_REF, TPEAK_REF, DELTA_AU,
                               [10e-9, 1e-6], [0.0, 1e-6, 40.1e-6], T0=295.0)
    assert np.allclose(T, 295.0, atol=1e-9)


def test_never_below_initial_temperature():
    times = np.geomspace(1e-9, 1e-4, 40)
    depths = np.linspace(0.0, 40.1e-6, 81)
    T = simulate_stack_heating(ENIG, F_REF, FWHM_REF, TPEAK_REF, DELTA_AU, times, depths)
    assert T.min() >= 300.0 - 1e-6


# 8. Interface: the thin Au film sits on Ni, whose thermal effusivity is lower
#    than gold's, so the ENIG surface ends the pulse hotter than bulk gold.
def test_nickel_underlayer_raises_surface_peak():
    times = np.linspace(20e-9, 40e-9, 41)
    T_enig = simulate_stack_heating(ENIG, F_REF, FWHM_REF, TPEAK_REF, DELTA_AU, times, [0.0])
    T_gold = simulate_stack_heating([dict(AU, thickness=40.1e-6)], F_REF, FWHM_REF,
                                    TPEAK_REF, DELTA_AU, times, [0.0])
    assert T_enig.max() > T_gold.max() + 10.0


# 9. Dense output grids (any time may coincide with an internal breakpoint of
#    a solver) must work and agree with a sparse request at shared times.
def test_dense_output_grid_consistent_with_sparse():
    dense = np.linspace(2e-9, 80e-9, 300)
    picks = [0, 75, 150, 299]
    T_dense = simulate_stack_heating(ENIG, F_REF, FWHM_REF, TPEAK_REF, DELTA_AU, dense, [0.0, 2e-6])
    T_sparse = simulate_stack_heating(ENIG, F_REF, FWHM_REF, TPEAK_REF, DELTA_AU, dense[picks], [0.0, 2e-6])
    assert np.all(np.isfinite(T_dense))
    rise = T_dense[picks] - 300.0
    assert np.max(np.abs(T_sparse - 300.0 - rise)) / np.max(rise) < 1e-4


# 10. Interface contract.
def test_output_shape_and_dtype():
    T = simulate_stack_heating(ENIG, F_REF, FWHM_REF, TPEAK_REF, DELTA_AU,
                               [10e-9, 20e-9, 30e-9], [0.0, 1e-6])
    assert isinstance(T, np.ndarray)
    assert T.shape == (3, 2)
    assert T.dtype == np.float64


@pytest.mark.parametrize("kwargs", [
    dict(layers=[]),
    dict(layers=[dict(AU, thickness=-1e-9)]),
    dict(layers=[{"thickness": 1e-6, "k": 1.0, "rho": 1.0}]),
    dict(absorbed_fluence=-1.0),
    dict(pulse_fwhm=0.0),
    dict(t_peak=1e-9),
    dict(penetration_depth=0.0),
    dict(t_out=[2e-8, 1e-8]),
    dict(t_out=[0.0, 1e-8]),
    dict(probe_depths=[-1e-9]),
    dict(probe_depths=[1.0]),
    dict(T0=0.0),
])
def test_invalid_inputs_raise(kwargs):
    args = dict(layers=ENIG, absorbed_fluence=F_REF, pulse_fwhm=FWHM_REF, t_peak=TPEAK_REF,
                penetration_depth=DELTA_AU, t_out=[1e-8], probe_depths=[0.0], T0=300.0)
    args.update(kwargs)
    with pytest.raises(ValueError):
        simulate_stack_heating(**args)
```